

Relatório

Reconhecimento Facial com Eigenfaces (Implementação via PCA)

Daniel Rossano Piccoli de Oliveira

DRE: 124174238

[Acessar Notebook no Google Colab](#)

Resumo

Esse relatório descreve a implementação de um sistema de reconhecimento facial simplificado baseado na técnica de Eigenfaces (Turk & Pentland, 1991), desenvolvida em Python. O objetivo do trabalho foi aplicar PCA para comprimir rostos num conjunto pequeno de números e usar essa representação comprimida para identificar pessoas. O dataset usado foi o *Olivetti Faces* com 40 pessoas e 10 fotos cada (com expressões, ângulos e iluminação ligeiramente diferentes em cada foto) totalizando 400 imagens. O núcleo matemático que envolveu o cálculo de componentes principais, projeção, reconstrução e classificação foi implementado a partir dos princípios aprendidos em CoCaDA sem uso de bibliotecas de alto nível que encapsulem o PCA. Cada seção deste relatório corresponde exatamente a uma seção do notebook do Colab onde o projeto foi apresentado.

1 Carregamento dos dados

Uma imagem digital em escala de cinza de dimensão 64×64 é, formalmente, uma função discreta $I : \{0, \dots, 63\} \times \{0, \dots, 63\} \rightarrow \mathbb{R}$. Para que os métodos de álgebra linear (autovalores, produto interno e normas) sejam aplicáveis, é necessário reinterpretar essa estrutura bidimensional como um vetor em $\mathbb{R}^{4096}$, ou seja, uma operação de *vetorização* (empilhamento das linhas ou colunas da imagem em sequência), que é uma bijeção linear e não descarta nenhuma informação, apenas reorganiza os mesmos 4096 valores escalares em uma estrutura de dado unidimensional.

Reunindo $n = 400$ dessas vetorizações como colunas, obtém-se a matriz de dados:

$$A \in \mathbb{R}^{4096 \times 400}, \quad A = \begin{bmatrix} a_1 & a_2 & \cdots & a_{400} \end{bmatrix}$$

onde cada coluna $a_i \in \mathbb{R}^{4096}$ é um rosto.

```
caminho_npz = "/content/olivetti_faces.npz"
dados_npz = np.load(caminho_npz)
imagens = dados_npz["imagens"]
```

```
rotulos = dados_npz["rotulos"]

altura, largura = imagens.shape[1], imagens.shape[2]
A = imagens.reshape(imagens.shape[0], -1).T
n_pixels, n_amstras = A.shape
n_pessoas = len(set(rotulos))
```

- `np.load(caminho_npz)` carrega um arquivo binário do Olivetti Faces já subido contendo o conjunto de imagens (`imagens`, array de forma $(400, 64, 64)$) e seus rótulos de identidade (`rotulos`, array de forma $(400,)$, com valores inteiros de 0 a 39 identificando cada uma das 40 pessoas).
- `imagens.reshape(imagens.shape[0], -1)` executa a vetorização descrita acima, transformando o array tridimensional $(400, 64, 64)$ em um array bidimensional $(400, 4096)$, onde a dimensão -1 é inferida automaticamente pelo NumPy como $64 \times 64 = 4096$.
- O `.T` (transposição) subsequente converte essa matriz de $(400, 4096)$ para $(4096, 400)$. Essa transposição garante que cada coluna, e não cada linha, represente um rosto.
- `n_pixels, n_amstras = A.shape` desempacota as dimensões resultantes $(4096$ e $400)$ para uso posterior em todo o notebook.
- `len(set(rotulos))` calcula o número de rótulos distintos, confirmando que existem 40 identidades no conjunto de dados.

2 Rosto médio e centralização

O Método das Componentes Principais busca as direções ortogonais de máxima variância de um conjunto de dados. Antes de aplicar o PCA, é preciso centralizar os dados, ou seja, calcular a média de cada pixel ao longo das 400 fotos e obter o "rosto médio" que é um rosto abstrato que representa a média entre todos os rostos. Depois se subtrai essa média de cada elemento da matriz. Isso é necessário porque o PCA busca direções de máxima variância dos dados e a variância é definida em relação à média. Sem centralizar, a direção de maior variação ficaria contaminada pela posição do centro de massa dos dados e não pela variação real entre os rostos. Com o rosto médio, qualquer variação a partir dele será uma característica individual de cada pessoa.

Define-se o rosto médio por:

$$\bar{a} = \frac{1}{n} \sum_{i=1}^n a_i \in \mathbb{R}^{4096}$$

e a matriz de dados centralizada:

$$A_c = A - \bar{a} \mathbf{1}_n^T, \quad \text{ou seja,} \quad (A_c)_{:,i} = a_i - \bar{a} \quad \forall i$$

```
media = A.mean(axis=1)
A_centralizada = A - media[:, None]
```

- `A.mean(axis=1)` calcula a média ao longo do eixo 1 (as colunas) para cada uma das 4096 linhas. Para cada pixel, calcula-se a média de seu valor através das 400 fotos. O resultado é o vetor $\bar{a}$, de forma `(4096,)`.
- `media[:, None]` é uma operação de reshape implícita que insere um eixo adicional, transformando o vetor de forma `(4096,)` em uma matriz-coluna de forma `(4096, 1)`. Essa transformação é necessária para que a subtração seguinte funcione corretamente. O NumPy expande automaticamente um array de forma `(4096, 1)` para operar elemento a elemento contra um array de forma `(4096, 400)`, subtraindo o vetor médio de cada uma das 400 colunas simultaneamente, sem necessidade de laços explícitos.
- O resultado, `A_centralizada`, é a A_c definida acima e é a matriz centralizada utilizada em toda a análise de componentes principais subsequente.

A célula correspondente no Colab também plota `media.reshape(altura, largura)`, remontando o vetor médio de volta à sua forma bidimensional original, mostrando o rosto médio apenas para fins de explanação visual, sem nenhuma implicação matemática adicional.

3 PCA via autovalores de $A^T A$

Seção central do notebook, tanto em complexidade teórica quanto em importância para a arquitetura de todo o sistema.

3.1 Problema de custo do PCA

Componentes principais são definidos como os autovetores da matriz de covariância amostral dos dados centralizados:

$$C = \frac{1}{n-1} A_c A_c^T \in \mathbb{R}^{4096 \times 4096}$$

É dividida a matriz $(n-1)$ para que os autovalores resultantes representem a variância estatística capturada por cada componente ao invés da soma dos quadrados da dispersão dos dados.

Resolver o problema de autovalores/autovetores diretamente sobre C exigiria diagonalizar uma matriz 4096×4096 , uma operação de alto custo em relação à dimensão $p = 4096$, computacionalmente excessiva nesse contexto.

3.2 Redução ao problema equivalente em $A^T A$

Como o número de amostras $n = 400$ é muito menor que a dimensão $p = 4096$, define-se a matriz alternativa muito menor:

$$M = \frac{1}{n-1} A_c^T A_c \in \mathbb{R}^{400 \times 400}$$

*Os autovetores não-nulos de $A_c^T A_c$ estão em correspondência bijetiva com os autovetores não-nulos de $A_c A_c^T$, através da aplicação de A_c , preservando o autovalor associado.

Demonstração: seja $v \in \mathbb{R}^{400}$ um autovetor de $A_c^T A_c$ associado ao autovalor λ , isto é:

$$A_c^T A_c v = \lambda v$$

Multiplicando ambos os lados dessa igualdade por A_c :

$$A_c (A_c^T A_c v) = A_c (\lambda v)$$

$$(A_c A_c^T) (A_c v) = \lambda (A_c v)$$

Essa última expressão afirma que o vetor $w = A_c v \in \mathbb{R}^{4096}$ satisfaz $A_c A_c^T w = \lambda w$, então w é autovetor de $A_c A_c^T$ associado ao mesmo autovalor λ . ■

O vetor $w = A_c v$ obtido pela proposição acima não possui, em geral, norma unitária, ainda que v tenha sido normalizado (a aplicação linear A_c não preserva norma). É necessário, portanto, normalizar explicitamente w após sua obtenção, para que constitua um autovetor no sentido próprio da palavra (com norma 1), adequado a servir como vetor de uma base ortonormal.

```
def calcular_pca(A, n_componentes=None):
    n amostras = A.shape[1]
    if n_componentes is None:
        n_componentes = n amostras

    M = (A.T @ A) / (n amostras - 1)
    autovalores, autovetores_M = np.linalg.eigh(M)

    ordem = np.argsort(autovalores)[::-1]
    autovalores = autovalores[ordem]
    autovetores_M = autovetores_M[:, ordem]

    autovetores = A @ autovetores_M
    normas = np.linalg.norm(autovetores, axis=0)
    normas[normas == 0] = 1
    autovetores = autovetores / normas

    n_componentes = min(n_componentes, autovetores.shape[1])
    return autovetores[:, :n_componentes], autovalores[:n_componentes]
```

- $M = (A.T @ A) / (n_amostras - 1)$: constrói explicitamente a matriz M de dimensão 400×400 . O parâmetro A dessa função recebe, na prática, a matriz $A_centralizada$ já centralizada.
- $np.linalg.eigh(M)$ retorna uma tupla $(autovalores, autovetores_M)$. Por convenção da biblioteca, `eigh` devolve os autovalores em ordem **crescente**; `autovetores_M` é uma matriz cujas colunas são os autovetores correspondentes de M .
- $ordem = np.argsort(autovalores)[::-1]$: `np.argsort` produz os índices que ordenariam o array de forma crescente, a fatia `[::-1]` inverte essa sequência de índices, produzindo a ordem decrescente desejada.

- `autovetores = A @ autovetores_M`: essa linha implementa a operação $w = A_c v$. Para cada coluna v de `autovetores_M`, calcula-se o produto matriz-vetor correspondente, recuperando os 400 autovetores (não-normalizados) de $A_c A_c^T$, cada um de dimensão 4096.
- `normas = np.linalg.norm(autovetores, axis=0)`: calcula a norma de cada coluna independentemente (`axis=0` percorre as linhas dentro de cada coluna fixa). A linha seguinte, `normas[normas == 0] = 1`, é uma proteção contra divisão por zero, para o caso teoricamente possível (mas improvável na prática com esse conjunto de dados).
- `autovetores = autovetores / normas`: normalização final, dividindo cada coluna pela sua respectiva norma, produzindo autovetores de norma unitária (as *eigenfaces*).
- As duas últimas linhas realizam a truncagem para os `n_componentes` solicitados. Se `n_componentes=None` foi fornecido, `n_componentes` foi redefinido como `n_amstras` (400) no início da função e a fatia final devolve, na prática, todos os componentes disponíveis (o `min` com `autovetores.shape[1]` é uma garantia adicional).

No fim, a célula faz

```
autovetores_completos, autovalores_completos = calcular_pca(
    A_centralizada, n_componentes=None)
```

obtendo a base completa de componentes principais que, na prática, tem exatamente 399 componentes não-triviais. A operação de centralização tem $\sum_i (A_c)_{:,i} = \mathbf{0}$ (a soma das colunas centralizadas é nula, pela média subtraída), o que reduz o posto máximo de A_c em uma unidade em relação ao número de colunas, de 400 para 399.

4 Visualização das eigenfaces

As eigenfaces são os autovetores finais da matriz de covariância remontados como imagens 64 por 64 revelando as direções de variações entre os rostos (como presença de óculos, iluminação, formato geral do rosto, etc.). As eigenfaces não são rostos reais, são apenas direções que capturam o máximo de variância estatística possível.

```
fig, eixos = plt.subplots(2, 5, figsize=(14, 6))
for i, eixo in enumerate(eixos.flat):
    eixo.imshow(autovetores_completos[:, i].reshape(altura, largura),
                cmap="gray")
    eixo.set_title(f"Eigenface {i+1}")
    eixo.axis("off")
```

Cada `autovetores_completos[:, i]` é um vetor de $\mathbb{R}^{4096}$ com norma 1. O `.reshape(altura, largura)` desfaz a vetorização descrita, remontando o vetor em sua forma matricial bidimensional original 64×64 para fins de visualização das imagens.

Cada eigenface representa uma direção ortogonal no espaço $\mathbb{R}^{4096}$ ao longo da qual os rostos do conjunto de treinamento exibem variação estatística decrescente (a primeira eigenface, correspondente ao maior autovalor, é a direção de maior variância e assim vai).

5 Variância explicada

Cada autovalor associado a uma eigenface representa quanta variância dos dados originais está concentrada naquela direção. Somando os autovalores dos primeiros k componentes e dividindo pela soma total, obtém-se a variância explicada acumulada. Essa métrica responde a pergunta de "quantos componentes é preciso manter para não perder muita informação". No experimento aplicado no código, cerca de 66 componentes já capturam 90% da variância total, 123 capturam 95% e 260 capturam 99% de um total de 399 componentes possíveis.

```
def variancia_explicada_acumulada(autovalores):
    autovalores = np.clip(autovalores, a_min=0, a_max=None)
    total = autovalores.sum()
    return np.cumsum(autovalores) / total if total > 0 else np.
        zeros_like(autovalores)
```

- `np.clip(autovalores, a_min=0, a_max=None)`: impõe um piso de zero aos autovalores. Teoricamente, todos os autovalores de uma matriz positiva (como M) são não-negativos. Essa linha é uma proteção contra autovalores ligeiramente negativos que podem surgir por erro de arredondamento de ponto flutuante em componentes cujo autovalor teórico é zero.
- `np.cumsum(autovalores)`: calcula a soma cumulativa até o componente de posição k .
- A divisão pelo `total` produz o vetor completo para todo k simultaneamente.

```
def n_componentes_para_variancia(autovalores, alvo):
    acumulada = variancia_explicada_acumulada(autovalores)
    indices = np.where(acumulada >= alvo)[0]
    return int(indices[0]) + 1 if len(indices) else len(autovalores)
```

Essa função resolve o problema inverso, ou seja, encontrar o menor k tal que a variância explicada seja maior ou igual ao alvo `np.where(acumulada >= alvo)[0]` retorna todos os índices que satisfazem a condição. Como o vetor `acumulada` é não-decrescente por construção (soma cumulativa de valores não-negativos), o primeiro desses índices, somado a 1 (para corrigir a 0-indexação do NumPy para a contagem de número de componentes) é a resposta.

6 Reconstrução com rank crescente

Dada a base ortonormal completa de eigenfaces $\{v_1, \dots, v_{399}\}$, qualquer rosto centralizado $a - \bar{a}$ admite a representação exata:

$$a - \bar{a} = \sum_{i=1}^{399} c_i v_i, \quad c_i = v_i^T (a - \bar{a})$$

por se tratar de uma expansão em base ortonormal completa do subespaço gerado pelos dados. A **aproximação de posto k** consiste em truncar esta soma nos k primeiros termos:

$$\hat{a}_k = \bar{a} + \sum_{i=1}^k c_i v_i = \bar{a} + V_k V_k^T (a - \bar{a})$$

onde $V_k = [v_1 \ \cdots \ v_k]$.

Um rosto pode ser aproximado usando só os k primeiros componentes. Projetar significa transformar um rosto de 4096 pixels em um vetor pequeno de k coeficientes, multiplicando pela transposta das eigenfaces. Reconstruir é o caminho inverso, multiplicar os coeficientes pelas eigenfaces e somar de volta a média, retornando um rosto aproximado de acordo com a quantidade de componentes usados.

O que o código faz

As funções `projetar` e `reconstruir`, definidas na Seção 3 do notebook, implementam exatamente as duas metades desta equação:

```
def projetar(A, media, autovetores):
    A_centralizada = A - media[:, None]
    return autovetores.T @ A_centralizada

def reconstruir(coeficientes, media, autovetores):
    return autovetores @ coeficientes + media[:, None]
```

`projetar` calcula $c = V_k^T(a - \bar{a})$: centraliza a entrada subtraindo a média recebida como parâmetro, e multiplica pela transposta da matriz de eigenfaces fornecida (`autovetores` ($4096, k$)), resultando em um vetor de coeficientes de forma $(k,)$ que é a representação comprimida do rosto.

`reconstruir` calcula $\hat{a} = V_k c + \bar{a}$, multiplicando os coeficientes pela matriz de eigenfaces (sem transpor, retornando ao espaço de 4096 dimensões) e soma de volta a média, desfazendo a centralização.

A função `projetar` já realiza a centralização internamente, subtraindo `media` de seu argumento de entrada. Consequentemente, todas as chamadas a `projetar` no notebook recebem como argumento os dados originais, não centralizados (por exemplo, `A` e não `A_centralizada`). Fornecer dados já centralizados resultaria em subtração da média duas vezes, distorcendo a representação comprimida resultante.

```
ranks = [1, 5, 50, 150, 299]
for r in ranks:
    coef = projetar(A[:, [indice_exemplo]], media,
                    autovetores_completos[:, :r])
    recon = reconstruir(coef, media, autovetores_completos[:, :r])
```

`autovetores_completos[:, :r]` é a operação que concretiza V_r que é a matriz contendo apenas as primeiras r colunas (eigenfaces) da base completa. É neste ponto que a redução de dimensionalidade ocorre. Ao aumentar r progressivamente (1, 5, 50, 150, 299), observa-se visualmente a reconstrução tornar-se mais fiel ao rosto original, com o rank máximo (299) produzindo reconstrução visualmente quase idêntica à original (o que já era esperado, pois $r = 299$ já captura a maioria da variância total (superior a 99%), estando muito próximo do posto máximo $n - 1 = 399$ que conteria a totalidade da informação representável pela base.

7 Erro de reconstrução vs. número de componentes

O Teorema de Eckart-Young-Mirsky estabelece que, para qualquer matriz A_c e qualquer $k \leq \text{posto}(A_c)$, a aproximação de posto k que minimiza o erro na norma de Frobenius,

$$\min_{\text{posto}(B) \leq k} \|A_c - B\|_F$$

é $B_k^* = V_k V_k^T A_c$ que é a projeção de A_c sobre o subespaço gerado pelos k primeiros componentes principais. Uma consequência desse teorema é que a função de erro é não crescente, então aumentar o número de componentes nunca vai piorar a melhor aproximação possível, apenas mantê-la ou melhorá-la.

```
erros = []
lista_ranks_erro = list(range(1, 51, 2))
for r in lista_ranks_erro:
    coef = projetar(A, media, autovetores_completos[:, :r])
    recon = reconstruir(coef, media, autovetores_completos[:, :r])
    erros.append(np.linalg.norm(A - recon, ord="fro"))
```

Para cada valor de r na sequência $\{1, 3, 5, \dots, 49\}$, a *matriz inteira* de 400 rostos é projetada e reconstruída e o erro é medido pela norma de Frobenius da diferença entre a matriz original e a reconstruída:

$$\|A - \hat{A}_r\|_F = \sqrt{\sum_{i,j} (A_{ij} - \hat{A}_{r,ij})^2}$$

```
decrecente = all(erros[i] >= erros[i+1] - 1e-6 for i in range(len(
    erros) - 1))
print("Erro progressivamente decrescente?", decrecente)
```

Essa linha realiza uma verificação computacional da propriedade teórica do Teorema de Eckart-Young: testa, para cada par de valores de rank consecutivos na sequência testada, se o erro não aumentou (com tolerância de 10^{-6}). O resultado **True** obtido na execução prova que a implementação está correta.

8 Classificação por vizinho mais próximo

O reconhecimento é resolvido como um problema de classificação de padrões pelo método do vizinho mais próximo: dado um vetor de consulta c_{teste} no espaço reduzido, atribui-se a ele o rótulo do vetor de treinamento $c_{\text{treino}}^{(j)}$ que minimiza a distância euclidiana:

$$j^* = \arg \min_j \left\| c_{\text{treino}}^{(j)} - c_{\text{teste}} \right\|_2$$

A avaliação de um classificador exige que o desempenho seja medido sobre dados que não participaram de seu ajuste. Se a base de eigenfaces fosse calculada utilizando todas as 400 fotos (incluindo aquelas escolhidas para teste), a estimativa de desempenho estaria contaminada por vazamento de dados, a base de componentes principais teria incluído as fotos de teste, aumentando artificialmente e invalidando a precisão relatada.

```
def classificar_vizinho_mais_proximo(coef_treino, rotulos_treino,
    coef_teste):
    previsoes = []
    for i in range(coef_teste.shape[1]):
        vetor_teste = coef_teste[:, i]
        distancias = np.linalg.norm(coef_treino - vetor_teste[:,
            None], axis=0)
        previsoes.append(rotulos_treino[np.argmin(distancias)])
    return np.array(previsoes)
```

Para cada coluna *i* de `coef_teste` (cada foto de teste projetada no espaço reduzido), calcula-se `coef_treino - vetor_teste[:, None]`: subtrai-se o vetor de teste (forma (k,1) após o `[:, None]`) de cada coluna de `coef_treino` (forma (k, n_treino)) simultaneamente, produzindo uma matriz de diferenças de mesma forma que `coef_treino`. `np.linalg.norm(..., axis=0)` calcula a norma euclidiana de cada coluna dessa matriz de diferenças independentemente, produzindo o vetor `distancias`, de forma (n_treino,), contendo a distância da foto de teste a cada uma das fotos de treino. `np.argmin(distancias)` localiza o índice de menor distância e o id da foto escolhida é anexado à lista de previsões.

```
idx_treino, idx_teste = train_test_split(
    indices, test_size=0.25, stratify=rotulos, random_state=42
)
```

`train_test_split` separa os 400 índices em dois subconjuntos disjuntos: 300 (75%) para treino e 100 (25%) para teste. O parâmetro `stratify=rotulos` garante que a proporção de cada uma das 40 classes seja preservada nos dois subconjuntos. `random_state=42` fixa a semente do gerador pseudoaleatório, permitindo que a divisão entre execuções distintas do notebook seja reproduzível.

```
media_treino = A_treino.mean(axis=1)
A_treino_centralizada = A_treino - media_treino[:, None]

n_max = min(A_treino.shape[1] - 1, 150)
autovetores_treino, _ = calcular_pca(A_treino_centralizada,
    n_componentes=n_max)
```

A média (`media_treino`) e a base de eigenfaces (`autovetores_treino`) são calculadas exclusivamente a partir de `A_treino`, nenhuma informação de `A_teste` participa desse cálculo. `n_max = min(A_treino.shape[1] - 1, 150)` limita o número de componentes calculados a no máximo 150 (por razões de custo computacional. O posto máximo de `A_treino_centralizada` seria de 299, mas apenas 150 são efetivamente necessários dado o comportamento de saturação já antecipado pela análise de variância explicada).

```
n_componentes_clf = 50
coef_treino = projetar(A_treino, media_treino, autovetores_treino
   [:, :n_componentes_clf])
coef_teste = projetar(A_teste, media_treino, autovetores_treino[:,
    :n_componentes_clf])
```

Ambas as chamadas de `projetar` usam `media_treino` e a mesma fatia de `autovetores_treino` que reutiliza os parâmetros aprendidos no treino sem jamais recalculá-los estatística nenhuma a partir do próprio conjunto de teste. Essa é a materialização em código da restrição: os

"parâmetros aprendidos" do sistema são exatamente o par $(\bar{a}_{\text{treino}}, V_{50})$, e o teste consiste em avaliar se a projeção de fotos não vistas durante este cálculo, usando esses parâmetros fixos, ainda preserva separabilidade suficiente entre as 40 classes para uma classificação por distância correta.

```
previsoes = classificar_vizinho_mais_proximo(coef_treino,
                                             rotulos_treino, coef_teste)
precisao = np.mean(previsoes == rotulos_teste)
```

`previsoes == rotulos_teste` produz um array booleano de comparação elemento a elemento. `np.mean` sobre um array booleano calcula a fração de valores True, resultando exatamente na taxa de acerto, a precisão da classificação. O resultado obtido nesta execução, com $k = 50$ componentes, foi de **94%**.

9 Precisão vs. número de componentes

Essa seção generaliza o experimento da seção anterior, testando o efeito da dimensão do espaço reduzido sobre a taxa de acerto:

```
lista_ranks_clf = sorted(set(r for r in [1, 2, 5, 10, 20, 30, 50,
                                         75, 100, n_max] if r <= n_max))
precisoos = []

for r in lista_ranks_clf:
    coef_tr = projetar(A_treino, media_treino, autovetores_treino
                      [:, :r])
    coef_te = projetar(A_teste, media_treino, autovetores_treino[:,
                                                                :r])
    preds = classificar_vizinho_mais_proximo(coef_tr,
                                             rotulos_treino, coef_te)
    precisoos.append(np.mean(preds == rotulos_teste))
```

Aqui o custo de recomputar o PCA para cada valor de r testado é evitado: a base completa `autovetores_treino` (calculada uma única vez até 150 componentes) é reutilizada (`autovetores_treino[:, :r]`) para cada valor de r na lista de ranks a testar, reaproveitando o resultado da decomposição espectral já obtida, e testando exclusivamente o efeito da redução de dimensionalidade sobre a classificação, sem custo adicional de re-diagonalização.

A precisão cresce rapidamente com poucos componentes e satura em torno de 20 a 30 componentes (evidência empírica de que a informação discriminativa entre identidades está concentrada nos componentes principais de maior variância), consistente com a análise de variância explicada e sugerindo que os componentes de ordem mais alta (associados a autovalores pequenos) contribuem predominantemente com ruído estatístico irrelevante para a tarefa de classificação, não com sinal discriminativo adicional.

10 Demonstração concreta de uma classificação

Aqui mostra-se o teste final da classificação, deixando explícito o mecanismo interno de decisão executado de forma agregada sobre as 100 fotos de teste simultaneamente.

```
def testar_uma_classificacao(indice_teste, A_treino, A_teste,
                             media_treino, autovetores,
                             rotulos_treino, rotulos_teste, altura,
                             largura, top_n=5):
    vetor_teste = projetar(A_teste[:, [indice_teste]], media_treino,
                           autovetores)[:, 0]
    coef_treino_local = projetar(A_treino, media_treino,
                                  autovetores)

    distancias = np.linalg.norm(coef_treino_local - vetor_teste[:,
                                                                    None], axis=0)
    ordem = np.argsort(distancias)
```

`A_teste[:, [indice_teste]]` seleciona uma única coluna de `A_teste` (o uso de uma lista de um único elemento, `[indice_teste]`, em vez do inteiro `indice_teste` diretamente, preserva a dimensão de coluna, mantendo o resultado como uma matriz de forma (4096, 1) ao invés de reduzi-lo a um vetor unidimensional, necessário para compatibilidade com a assinatura de `projetar` que espera uma matriz). O restante da função reaplica a lógica de `classificar_vizinho_mais_proximo`, mas retendo, através de `np.argsort(distancias)`, toda a ordenação das distâncias, e não apenas o índice de mínimo, permitindo reportar também as `top_n` fotos de treinamento mais próximas, com suas respectivas distâncias numéricas e uma visualização lado a lado que evidencia visualmente a plausibilidade (ou implausibilidade) da decisão do classificador. O parâmetro `autovetores` recebido por essa função, na chamada realizada na célula correspondente, é `autovetores_treino[:, :n_componentes_clf]`, ou seja, a mesma base de 50 eigenfaces utilizada na classificação agregada, garantindo que a demonstração individual seja fielmente representativa do experimento estatístico completo.

11 Conclusões

A implementação demonstra que:

1. A matriz de covariância de um conjunto de dados de alta dimensão pode ser diagonalizada de forma computacionalmente eficiente através da matriz reduzida $A^T A$, quando o número de amostras é significativamente menor que a dimensionalidade dos dados (resultado justificado pela relação de autovalores/autovetores entre $A^T A$ e AA^T).
2. A normalização da covariância por $(n-1)$, e não por n , decorre da correção de Bessel para estimadores não-viesados de variância, sem afetar a geometria das eigenfaces em si.
3. A aproximação de posto k obtida por truncagem da base de componentes principais é, pelo Teorema de Eckart-Young-Mirsky, a melhor aproximação possível na norma de Frobenius.
4. Uma redução de dimensionalidade de 4096 pixels para 50 coeficientes preserva informação suficiente para uma tarefa de classificação por vizinho mais próximo com 94% de precisão.